

EXPERIMENT 5 - ASSIGNMENT 2

Title

Study and Implementation of AWS RDS and NoSQL Databases: Deploy and Connect DynamoDB for Flask Application.

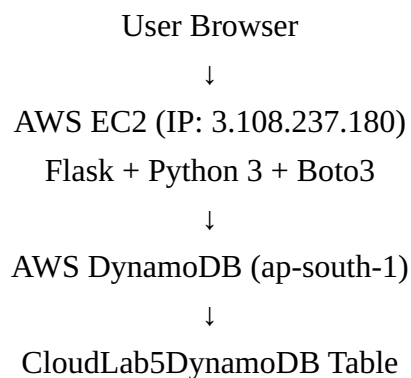
Aim

To deploy an AWS DynamoDB NoSQL database, connect it to a Python Flask application running on an EC2 instance, and successfully demonstrate Create, Read, Update, and Delete (CRUD) operations.

Objective

The objective is to configure an EC2 instance to run a Python Flask web application using the boto3 SDK, securely connect it to a managed AWS DynamoDB table via IAM roles, and provide concrete evidence of database functionality through full CRUD operations using both the AWS CLI and the frontend interface.

1. ARCHITECTURE

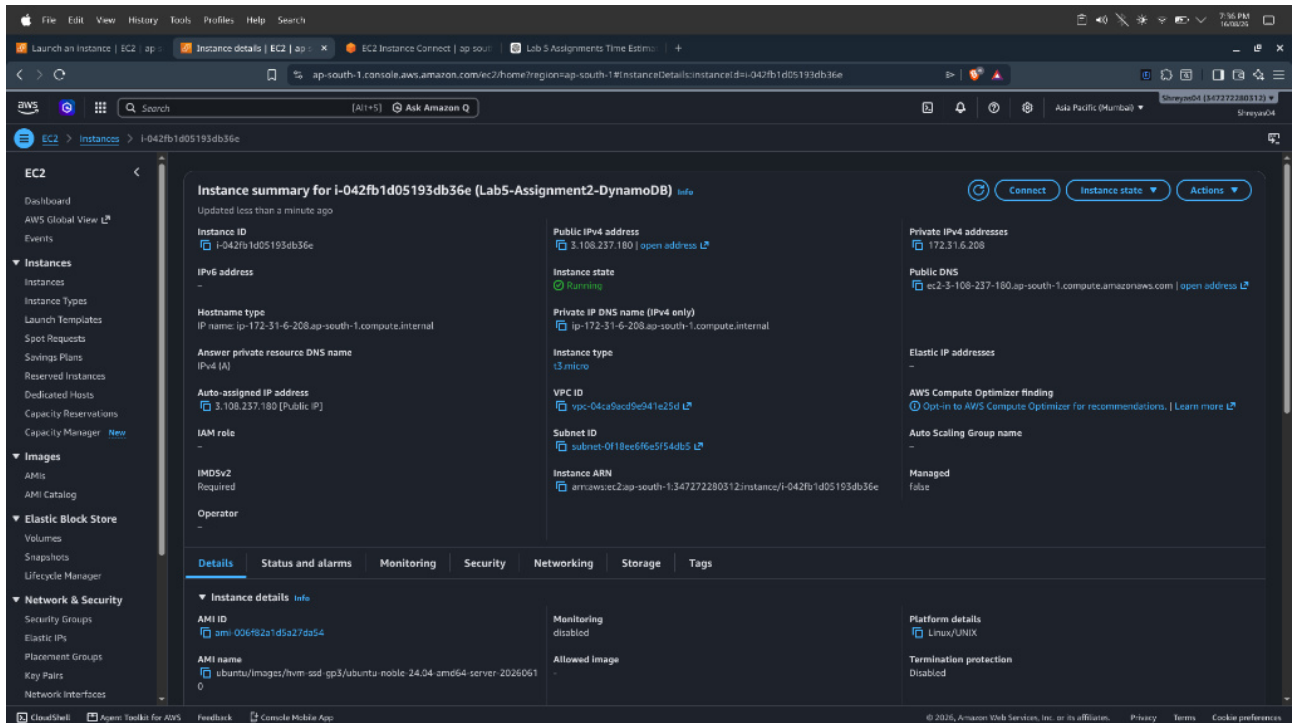


The architecture involves an EC2 instance acting as the web server hosting a Python Flask application on port 5000. The application securely connects to an AWS DynamoDB table named 'CloudLab5DynamoDB' using an attached IAM Role, avoiding the need to hardcode credentials.

2. STEP-BY-STEP IMPLEMENTATION

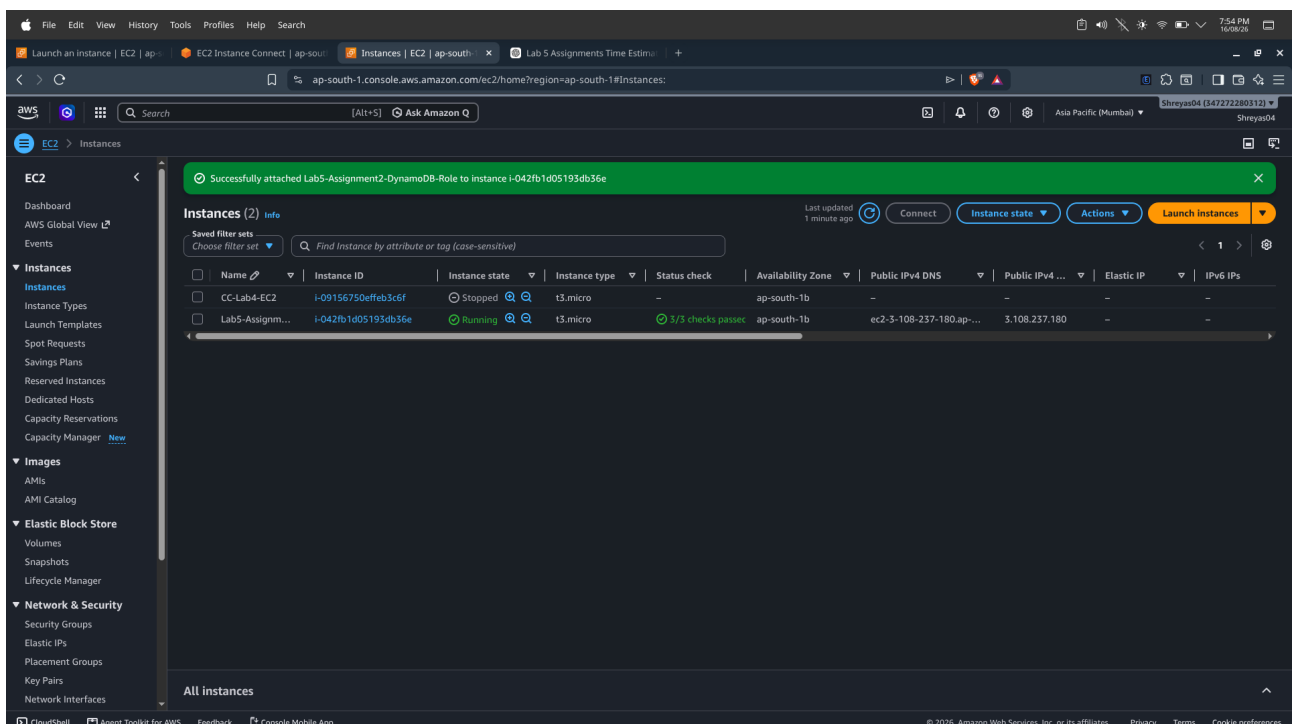
Step 1: EC2 Instance Provisioning

An EC2 instance named 'Lab5-Assignment2-DynamoDB' was successfully launched and accessed. The public IP is 3.108.237.180.



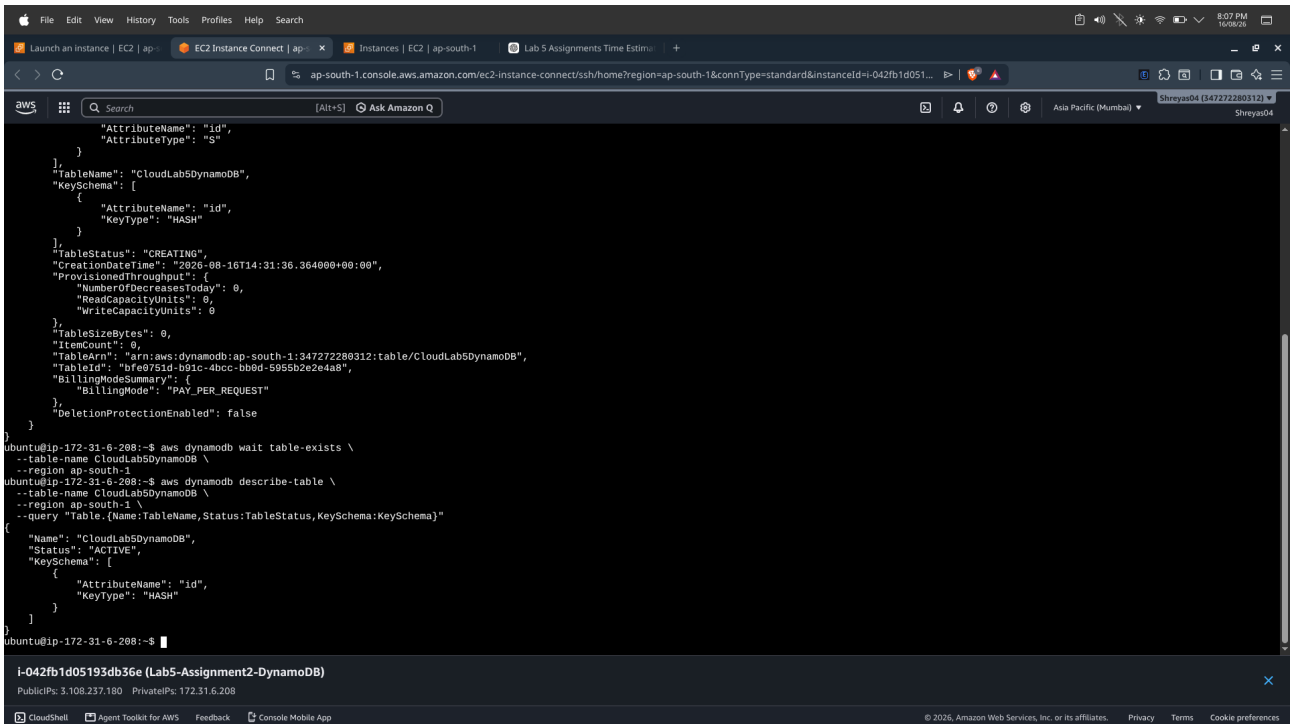
Step 2: IAM Role Configuration

To securely connect EC2 with DynamoDB, an IAM Role 'Lab5-Assignment2-DynamoDB-Role' was successfully attached to the instance.



Step 3: DynamoDB Table Creation

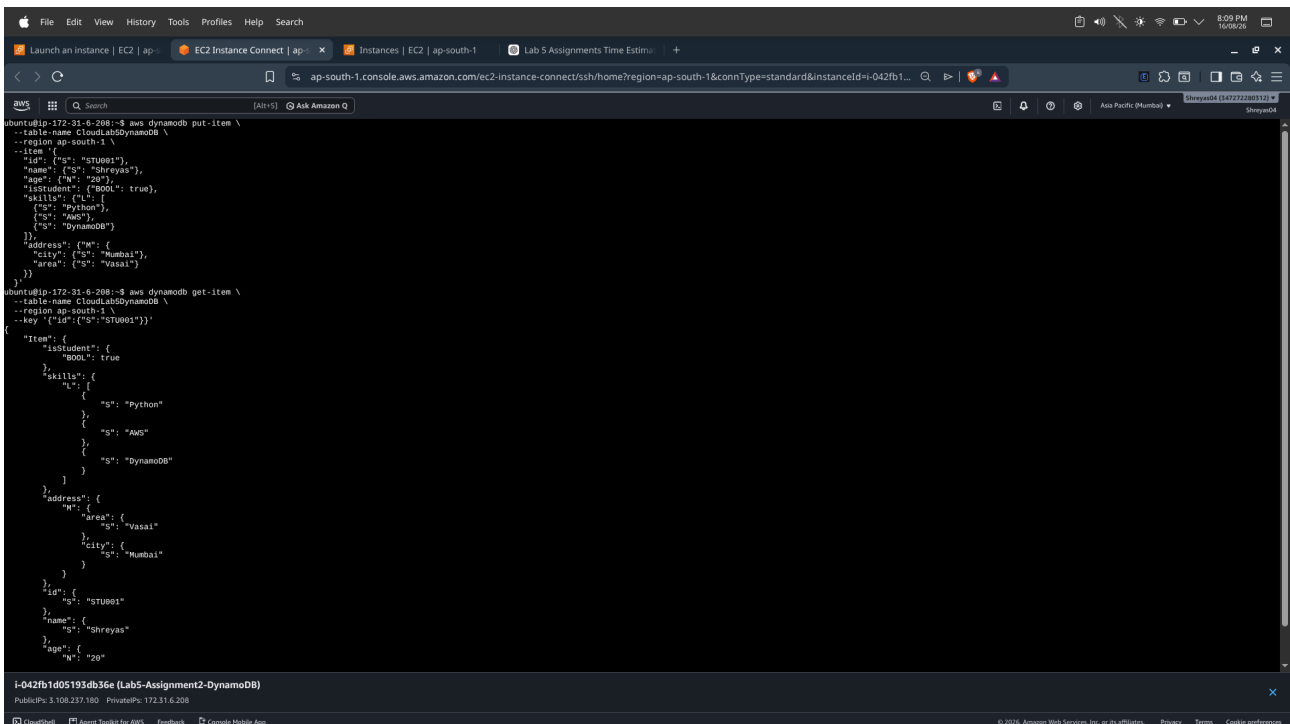
The 'CloudLab5DynamoDB' table was created successfully using the AWS CLI, verifying the 'ACTIVE' status and HASH key schema.



```
awscli@ip-172-31-6-208:~$ aws dynamodb create-table \
--table-name CloudLab5DynamoDB \
--region ap-south-1 \
--key-schema '[{"AttributeName": "id", "KeyType": "HASH"}]' \
--provisioned-throughput '{"ReadCapacityUnits": 0, "WriteCapacityUnits": 0}' \
--billing-mode PAY_PER_REQUEST \
--deletion-protection-enabled false
{
  "TableName": "CloudLab5DynamoDB",
  "KeySchema": [
    {
      "AttributeName": "id",
      "KeyType": "HASH"
    }
  ],
  "TableStatus": "CREATING",
  "CreationDateTime": "2026-08-16T14:31:36.364000+00:00",
  "ProvisionedThroughput": {
    "NumberOfDecreasesToday": 0,
    "ReadCapacityUnits": 0,
    "WriteCapacityUnits": 0
  },
  "TableSizeBytes": 0,
  "ItemCount": 0,
  "TableArn": "arn:aws:dynamodb:ap-south-1:347272280312:table/CloudLab5DynamoDB",
  "TableId": "tfe6751d-bb1c-4bcc-bb6d-5955b2e2e4a8",
  "BillingModeSummary": {
    "BillingMode": "PAY_PER_REQUEST"
  },
  "DeletionProtectionEnabled": false
}
awscli@ip-172-31-6-208:~$ aws dynamodb wait table-exists \
--table-name CloudLab5DynamoDB \
--region ap-south-1
awscli@ip-172-31-6-208:~$ aws dynamodb describe-table \
--table-name CloudLab5DynamoDB \
--region ap-south-1 \
--query 'Table.{Name:TableName,Status:TableStatus,KeySchema:KeySchema}'
{
  "Name": "CloudLab5DynamoDB",
  "Status": "ACTIVE",
  "KeySchema": [
    {
      "AttributeName": "id",
      "KeyType": "HASH"
    }
  ]
}
```

Step 4: Database CRUD via CLI - Create & Read

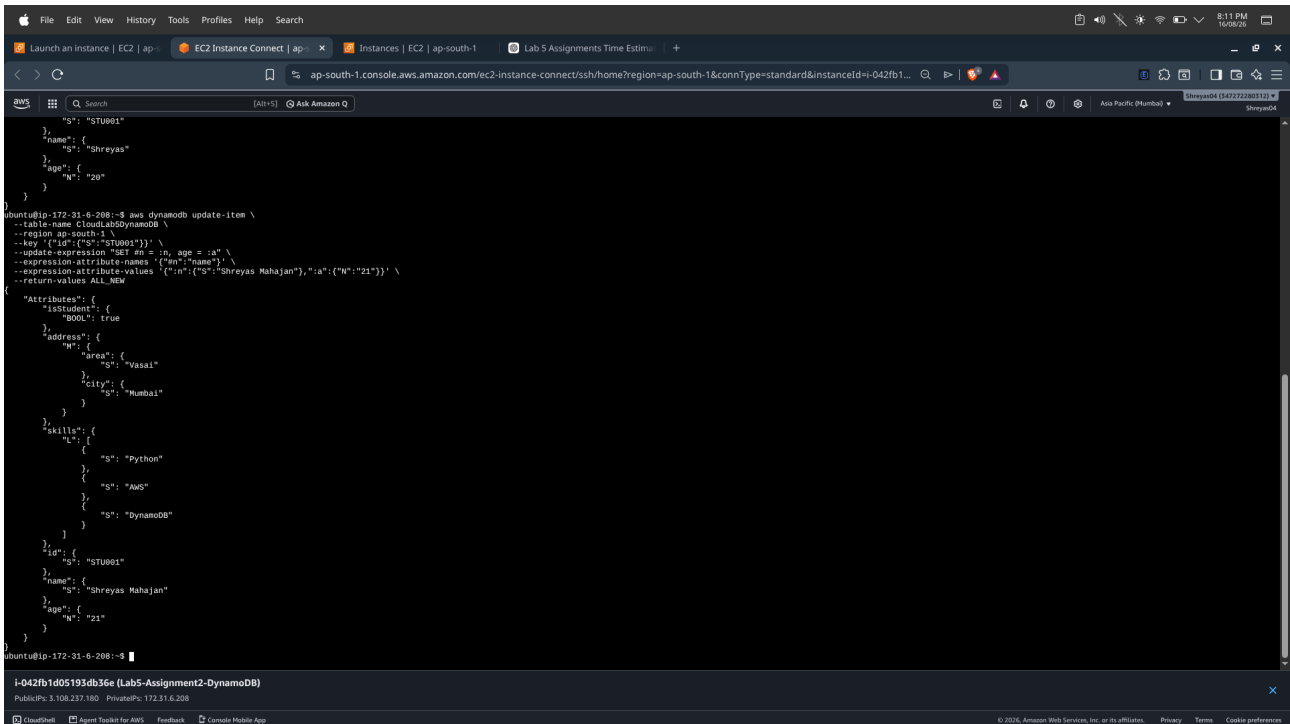
A new item (STU001, Shreyas) was added using 'put-item' and retrieved successfully using 'get-item' in the CLI.



```
awscli@ip-172-31-6-208:~$ aws dynamodb put-item \
--table-name CloudLab5DynamoDB \
--region ap-south-1 \
--item '{
  "id": {"S": "STU001"},
  "name": {"S": "Shreyas"},
  "age": {"N": "20"},
  "isStudent": {"BOOL": true},
  "skills": {"L": [
    {"S": "Python"},
    {"S": "AWS"},
    {"S": "DynamoDB"}
  ]},
  "address": {"M": {
    "city": {"S": "Mumbai"},
    "area": {"S": "Vasai"}
  }}
}'
awscli@ip-172-31-6-208:~$ aws dynamodb get-item \
--table-name CloudLab5DynamoDB \
--region ap-south-1 \
--key '{"id":{"S":"STU001"}}'
{
  "Item": {
    "id": {
      "S": "STU001"
    },
    "name": {
      "S": "Shreyas"
    },
    "age": {
      "N": "20"
    },
    "isStudent": {
      "BOOL": true
    },
    "skills": {
      "L": [
        {
          "S": "Python"
        },
        {
          "S": "AWS"
        },
        {
          "S": "DynamoDB"
        }
      ]
    },
    "address": {
      "M": {
        "area": {
          "S": "Vasai"
        },
        "city": {
          "S": "Mumbai"
        }
      }
    }
  }
}
```

Step 5: Database CRUD via CLI - Update

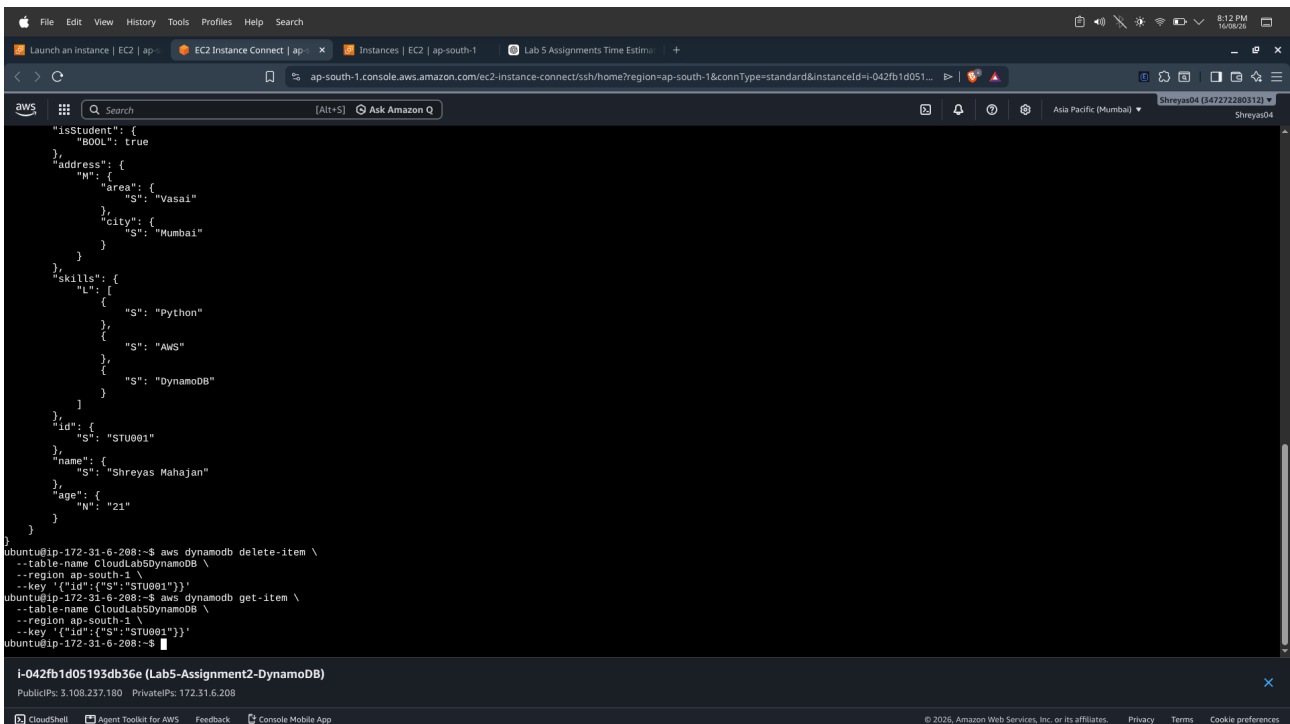
The student record was updated using 'update-item' to change the name to 'Shreyas Mahajan' and age to '21'.



```
awscli@ip-172-31-6-208:~$ aws dynamodb update-item \
--table-name CloudLab5DynamoDB \
--region ap-south-1 \
--key '{"id":{"S":"STU001"}}' \
--update-expression "SET #n = :n, age = :a" \
--expression-attribute-names {"#n":"name"} \
--expression-attribute-values [{"#n":{"S":"Shreyas Mahajan"}}, {"a":{"N":"21"}}] \
--return-values ALL_NEW
{
  "Attributes": {
    "isStudent": {
      "BOOL": true
    },
    "address": {
      "M": {
        "area": {
          "S": "Vasai"
        },
        "city": {
          "S": "Mumbai"
        }
      }
    },
    "skills": {
      "L": [
        {
          "S": "Python"
        },
        {
          "S": "AWS"
        },
        {
          "S": "DynamoDB"
        }
      ]
    },
    "id": {
      "S": "STU001"
    },
    "name": {
      "S": "Shreyas Mahajan"
    },
    "age": {
      "N": "21"
    }
  }
}
```

Step 6: Database CRUD via CLI - Delete

The item was deleted using 'delete-item', and verified with 'get-item' which returned an empty response.

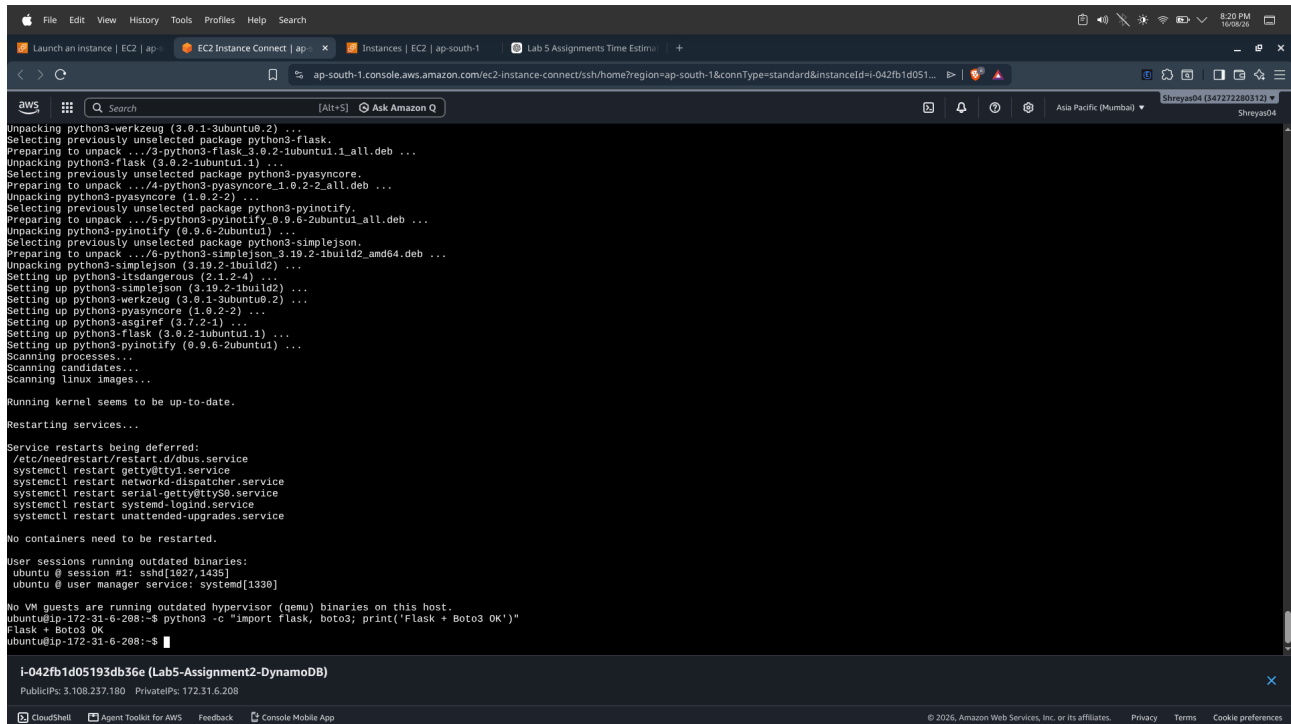


```
awscli@ip-172-31-6-208:~$ aws dynamodb delete-item \
--table-name CloudLab5DynamoDB \
--region ap-south-1 \
--key '{"id":{"S":"STU001"}}'
{
  "Deleted": true
}

awscli@ip-172-31-6-208:~$ aws dynamodb get-item \
--table-name CloudLab5DynamoDB \
--region ap-south-1 \
--key '{"id":{"S":"STU001"}}'
{}
```

Step 7: Application Dependencies Setup

Installed required Python packages (Flask, werkzeug, etc.) and successfully tested the import of 'flask' and 'boto3'.



```
Unpacking python3-werkzeug (3.0.1-2ubuntu0.2) ...
Selecting previously unselected package python3-flask.
Preparing to unpack .../3-python3-flask_3.0.2-1ubuntu1.1_all.deb ...
Unpacking python3-flask (3.0.2-1ubuntu1.1) ...
Selecting previously unselected package python3-pyasyncore.
Preparing to unpack .../4-python3-pyasyncore_1.0.2-2_all.deb ...
Unpacking python3-pyasyncore (1.0.2-2) ...
Selecting previously unselected package python3-pynotify.
Preparing to unpack .../5-python3-pynotify_0.9.6-2ubuntu1_all.deb ...
Unpacking python3-pynotify (0.9.6-2ubuntu1) ...
Selecting previously unselected package python3-simplejson.
Preparing to unpack .../6-python3-simplejson_3.19.2-1build2_amd64.deb ...
Unpacking python3-simplejson (3.19.2-1build2) ...
Setting up python3-itsdangerous (2.1.2-4) ...
Setting up python3-simplejson (3.19.2-1build2) ...
Setting up python3-werkzeug (3.0.1-3ubuntu0.2) ...
Setting up python3-pyasyncore (1.0.2-2) ...
Setting up python3-asgiref (3.7.2-1) ...
Setting up python3-flask (3.0.2-1ubuntu1.1) ...
Setting up python3-pynotify (0.9.6-2ubuntu1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...

Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart getty@tty1.service
systemctl restart networkd-dispatcher.service
systemctl restart serial-getty@ttyS0.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

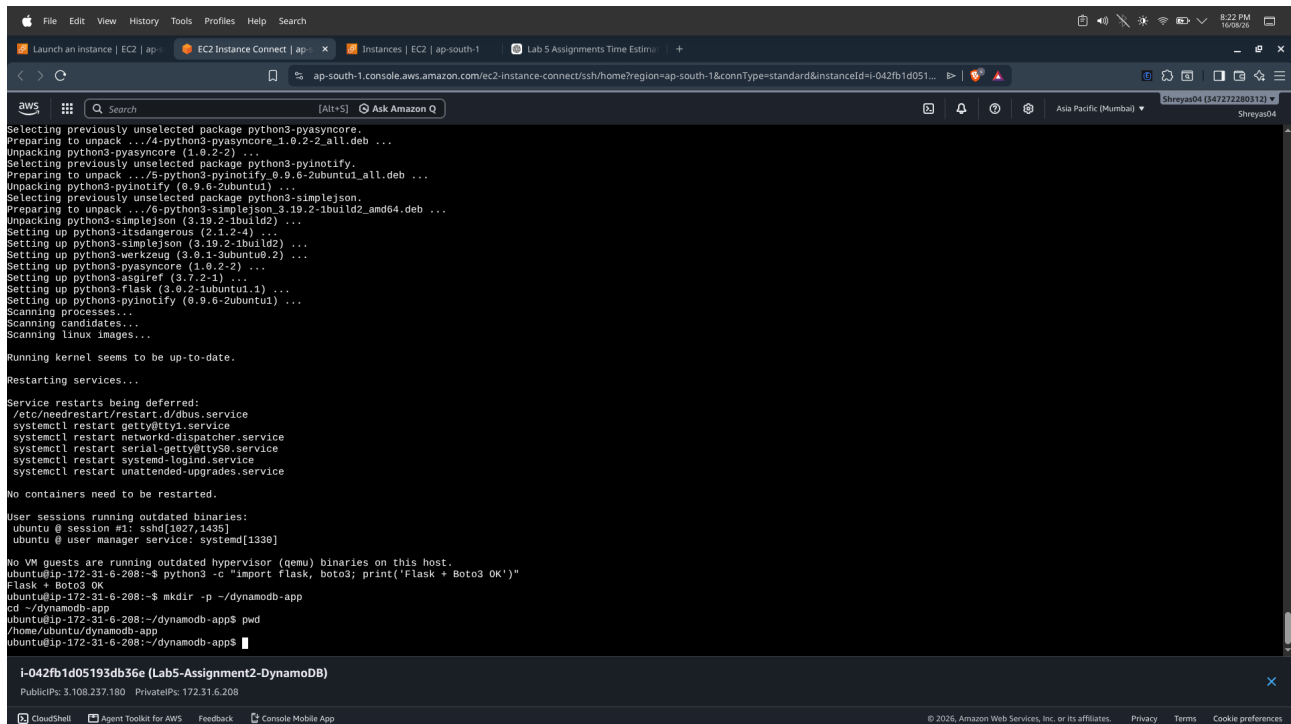
No containers need to be restarted.

User sessions running outdated binaries:
ubuntu @ session #1: sshd[1027,1435]
ubuntu @ user manager service: systemd[1330]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-6-208:~$ python3 -c "import flask, boto3; print('Flask + Boto3 OK!')"
Flask + Boto3 OK
ubuntu@ip-172-31-6-208:~$
```

Step 8: Application Directory Creation

Created the project directory '~/dynamodb-app' to house the application files.



```
Selecting previously unselected package python3-pyasyncore.
Preparing to unpack .../4-python3-pyasyncore_1.0.2-2_all.deb ...
Unpacking python3-pyasyncore (1.0.2-2) ...
Selecting previously unselected package python3-pynotify.
Preparing to unpack .../5-python3-pynotify_0.9.6-2ubuntu1_all.deb ...
Unpacking python3-pynotify (0.9.6-2ubuntu1) ...
Selecting previously unselected package python3-simplejson.
Preparing to unpack .../6-python3-simplejson_3.19.2-1build2_amd64.deb ...
Unpacking python3-simplejson (3.19.2-1build2) ...
Setting up python3-itsdangerous (2.1.2-4) ...
Setting up python3-simplejson (3.19.2-1build2) ...
Setting up python3-werkzeug (3.0.1-3ubuntu0.2) ...
Setting up python3-pyasyncore (1.0.2-2) ...
Setting up python3-asgiref (3.7.2-1) ...
Setting up python3-flask (3.0.2-1ubuntu1.1) ...
Setting up python3-pynotify (0.9.6-2ubuntu1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...

Service restarts being deferred:
/etc/needrestart/restart.d/dbus.service
systemctl restart getty@tty1.service
systemctl restart networkd-dispatcher.service
systemctl restart serial-getty@ttyS0.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

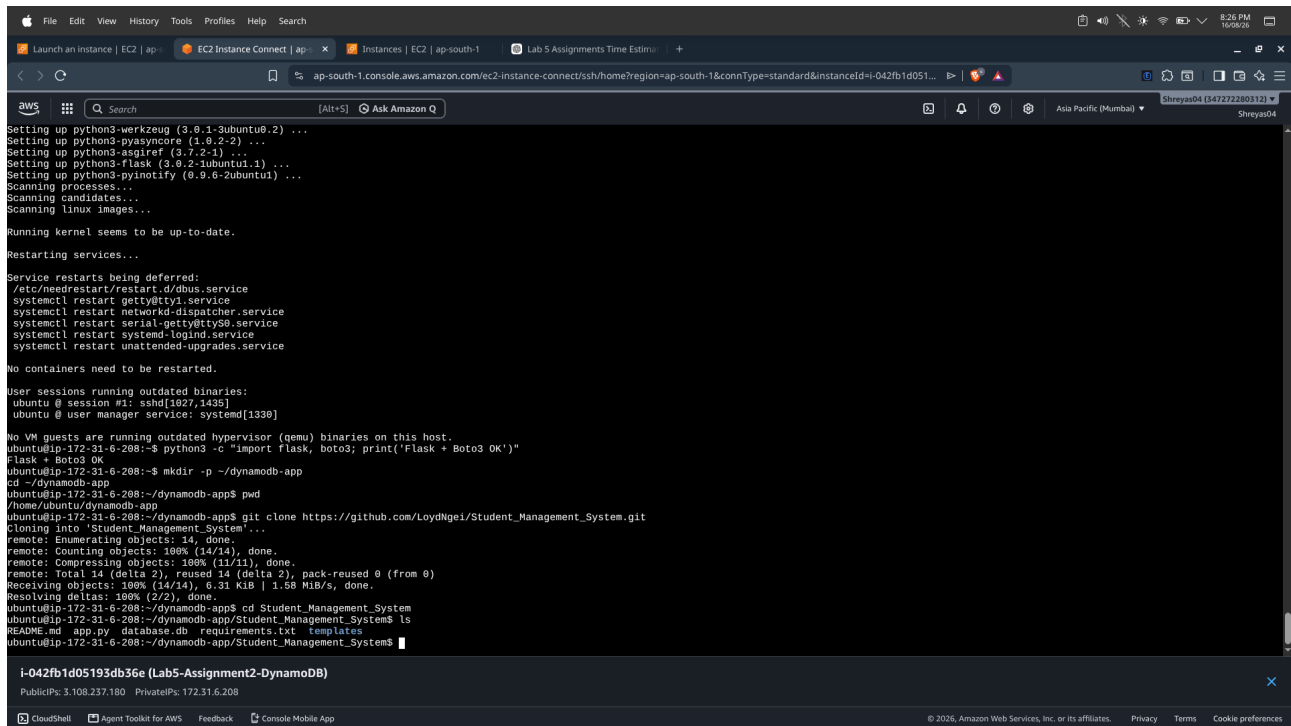
No containers need to be restarted.

User sessions running outdated binaries:
ubuntu @ session #1: sshd[1027,1435]
ubuntu @ user manager service: systemd[1330]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-6-208:~$ python3 -c "import flask, boto3; print('Flask + Boto3 OK!')"
Flask + Boto3 OK
ubuntu@ip-172-31-6-208:~$ mkdir -p ~/dynamodb-app
cd ~/dynamodb-app
ubuntu@ip-172-31-6-208:~/dynamodb-app$ pwd
/home/ubuntu/dynamodb-app
ubuntu@ip-172-31-6-208:~/dynamodb-app$
```

Step 9: Cloning Application Repository

Cloned the open-source Flask project 'Student_Management_System' into the EC2 directory.



```
Setting up python3-werkzeug (3.0.1-3ubuntu0.2) ...
Setting up python3-pyasyncore (1.0.2-2) ...
Setting up python3-asgiref (3.7.2-1) ...
Setting up python3-flask (3.0.2-1ubuntu1) ...
Setting up python3-pyjwt (2.8.0-2ubuntu1) ...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

Restarting services...

Service restarts being deferred:
/etc/needrestart/start.d/dbus.service
systemctl restart getty@tty1.service
systemctl restart networkd-dispatcher.service
systemctl restart serial-getty@tty08.service
systemctl restart systemd-logind.service
systemctl restart unattended-upgrades.service

No containers need to be restarted.

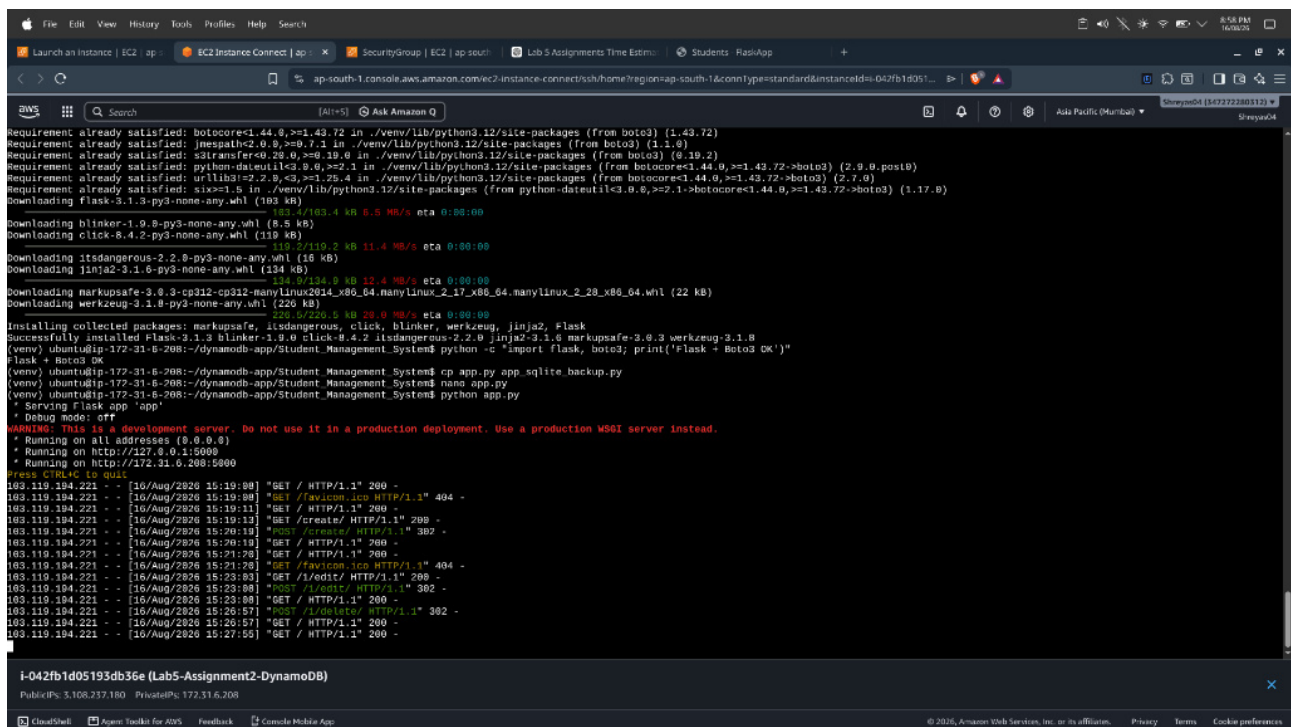
User sessions running outdated binaries:
ubuntu @ session #1: sshd[6027,1430]
ubuntu @ user manager service: systemd[1330]

No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-6-208:~$ python3 -c "import flask, boto3; print('Flask + Boto3 OK!')"
Flask + Boto3 OK
ubuntu@ip-172-31-6-208:~$ mkdir -p ~/dynamodb-app
cd ~/dynamodb-app
ubuntu@ip-172-31-6-208:~/dynamodb-app$ pwd
/home/ubuntu/dynamodb-app
ubuntu@ip-172-31-6-208:~/dynamodb-app$ git clone https://github.com/LoydNgei/Student_Management_System.git
Cloning into 'Student_Management_System'...
remote: Enumerating objects: 14, done.
remote: Counting objects: 100% (14/14), done.
remote: Compressing objects: 100% (11/11), done.
remote: Total 14 (delta 2), reused 14 (delta 2), pack-reused 0 (from 0)
Receiving objects: 100% (14/14), 6.31 KiB | 1.58 MiB/s, done.
Resolving deltas: 100% (2/2), done.
ubuntu@ip-172-31-6-208:~/dynamodb-app$ cd Student_Management_System
ubuntu@ip-172-31-6-208:~/dynamodb-app/Student_Management_System$ ls
README.md  app.py  database.db  requirements.txt  templates
ubuntu@ip-172-31-6-208:~/dynamodb-app/Student_Management_System$
```

i-042fb1d05193db36e (Lab5-Assignment2-DynamoDB)
PublicIPs: 3.108.237.180 PrivateIPs: 172.31.6.208

Step 10: Running Flask Application

Started the Flask server on port 5000. Real-time HTTP GET and POST requests are visible in the terminal log.

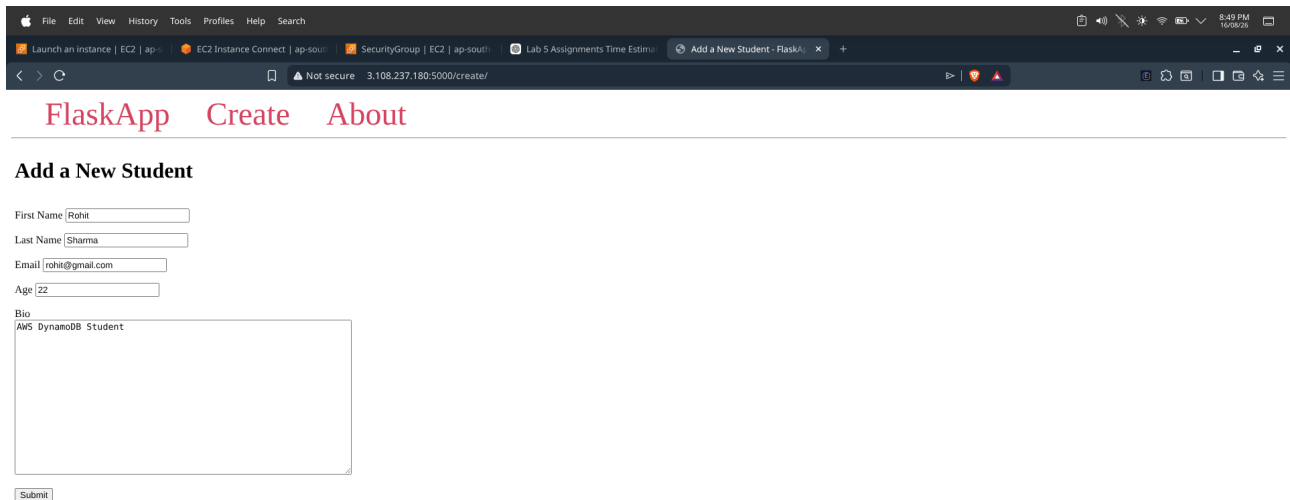


```
Requirement already satisfied: boto3 in /home/ubuntu/.local/lib/python3.12/site-packages (from boto3) (1.43.72)
Requirement already satisfied: jmespath<2.0.0,>=0.7.1 in /home/ubuntu/.local/lib/python3.12/site-packages (from boto3) (1.1.0)
Requirement already satisfied: s3transfer<0.28.0,>=0.19.0 in /home/ubuntu/.local/lib/python3.12/site-packages (from boto3) (0.19.2)
Requirement already satisfied: python-dateutil<3.0.0,>=2.1 in /home/ubuntu/.local/lib/python3.12/site-packages (from boto3) (2.9.0.post0)
Requirement already satisfied: urllib3<2.0.0,>=1.25.4 in /home/ubuntu/.local/lib/python3.12/site-packages (from boto3) (2.2.3)
Requirement already satisfied: six<1.16.0,>=1.10 in /home/ubuntu/.local/lib/python3.12/site-packages (from python-dateutil<3.0.0,>=2.1->boto3) (1.17.0)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
103.4/103.4 kB 5.5 MB/s eta 0:00:00
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
8.5/8.5 kB 11.4 MB/s eta 0:00:00
Downloading click-8.4.2-py3-none-any.whl (110 kB)
110.2/110.2 kB 12.4 MB/s eta 0:00:00
Downloading Jinja2-3.1.6-py3-none-any.whl (134 kB)
134.0/134.0 kB 12.4 MB/s eta 0:00:00
Downloading markupsafe-3.0.3-cp312-cp312-manylinux2014_x86_64_manylinux_2_17_x86_64_manylinux_2_28_x86_64.whl (22 kB)
22.0/22.0 kB 12.4 MB/s eta 0:00:00
Downloading werkzeug-3.1.2-py3-none-any.whl (226 kB)
226.5/226.5 kB 20.9 MB/s eta 0:00:00
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, Jinja2, Flask
Successfully installed Flask-3.1.3 blinker-1.9.0 click-8.4.2 itsdangerous-2.2.0 Jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.2
(venv) ubuntu@ip-172-31-6-208:~/dynamodb-app/Student_Management_System$ python -c "import flask, boto3; print('Flask + Boto3 OK!')"
Flask + Boto3 OK
(venv) ubuntu@ip-172-31-6-208:~/dynamodb-app/Student_Management_System$ cp app.py app_sqlite_backup.py
(venv) ubuntu@ip-172-31-6-208:~/dynamodb-app/Student_Management_System$ nano app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.6.208:5000
Press CTRL+C to quit
103.119.194.221 - - [16/Aug/2026 15:19:08] "GET / HTTP/1.1" 200 -
103.119.194.221 - - [16/Aug/2026 15:19:08] "GET /favicon.ico HTTP/1.1" 404 -
103.119.194.221 - - [16/Aug/2026 15:19:11] "GET / HTTP/1.1" 200 -
103.119.194.221 - - [16/Aug/2026 15:19:13] "GET /create/ HTTP/1.1" 200 -
103.119.194.221 - - [16/Aug/2026 15:20:10] "POST /create/ HTTP/1.1" 302 -
103.119.194.221 - - [16/Aug/2026 15:20:19] "GET / HTTP/1.1" 200 -
103.119.194.221 - - [16/Aug/2026 15:21:28] "GET / HTTP/1.1" 200 -
103.119.194.221 - - [16/Aug/2026 15:21:28] "GET /favicon.ico HTTP/1.1" 404 -
103.119.194.221 - - [16/Aug/2026 15:23:03] "GET /static/ HTTP/1.1" 200 -
103.119.194.221 - - [16/Aug/2026 15:23:08] "POST /login/ HTTP/1.1" 302 -
103.119.194.221 - - [16/Aug/2026 15:23:08] "GET / HTTP/1.1" 200 -
103.119.194.221 - - [16/Aug/2026 15:26:57] "POST /logout/ HTTP/1.1" 302 -
103.119.194.221 - - [16/Aug/2026 15:26:57] "GET / HTTP/1.1" 200 -
103.119.194.221 - - [16/Aug/2026 15:27:55] "GET / HTTP/1.1" 200 -
```

i-042fb1d05193db36e (Lab5-Assignment2-DynamoDB)
PublicIPs: 3.108.237.180 PrivateIPs: 172.31.6.208

Step 11: Web Application Frontend - Create

Accessed the web UI via the public IP to add a new student (Rohit Sharma, Age 22).



The screenshot shows a web browser window with the URL `3.108.237.180:5000/create/`. The page has a navigation bar with links for [FlaskApp](#), [Create](#), and [About](#). The main heading is **Add a New Student**. The form contains the following fields:

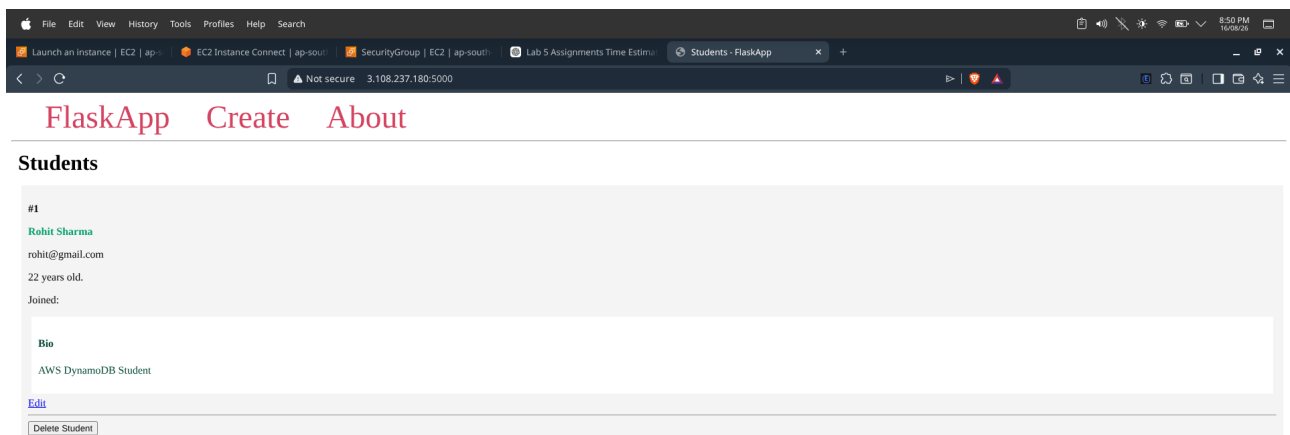
- First Name:
- Last Name:
- Email:
- Age:
- Bio:

AWS DynamoDB Student

A **Submit** button is located at the bottom left of the form.

Step 12: Web Application Frontend - Read

The added student record successfully displayed under the Students list, pulling data from DynamoDB.



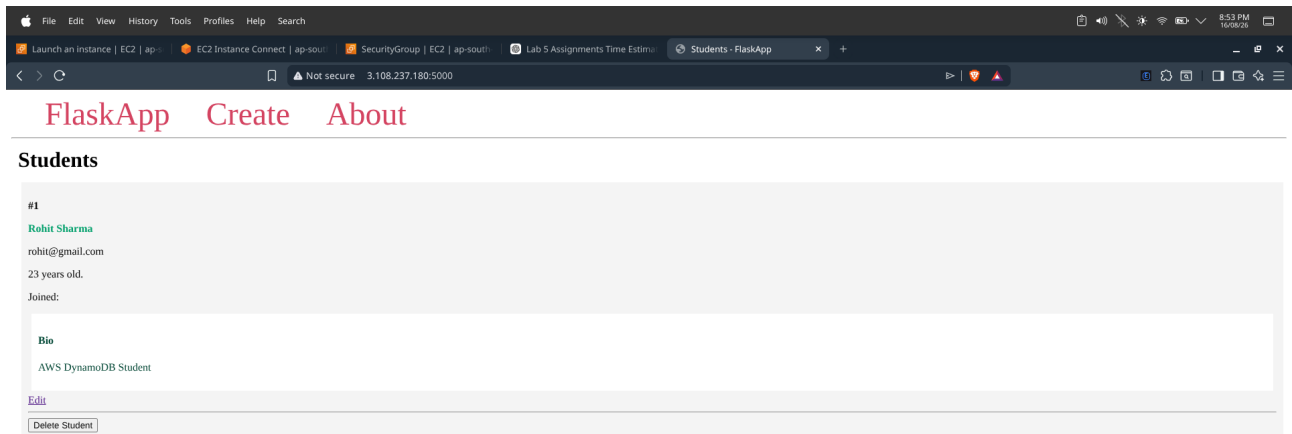
The screenshot shows a web browser window with the URL `3.108.237.180:5000`. The page has a navigation bar with links for [FlaskApp](#), [Create](#), and [About](#). The main heading is **Students**. A list of students is displayed, with the first entry being:

- #1
- Rohit Sharma**
- rohit@gmail.com
- 22 years old.
- Joined:

Below the student information is a **Bio** section containing the text `AWS DynamoDB Student`. At the bottom of the entry, there are two links: [Edit](#) and [Delete Student](#).

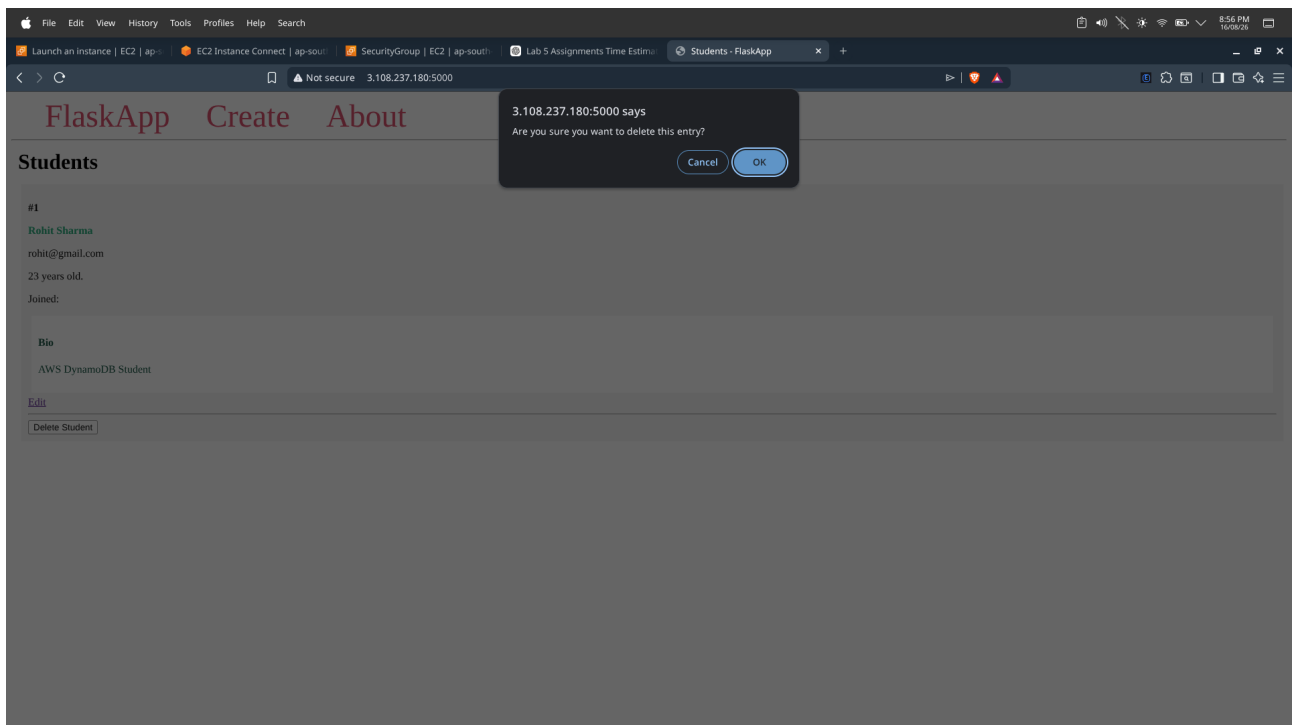
Step 13: Web Application Frontend - Update

Updated the student's age from 22 to 23 via the application interface.



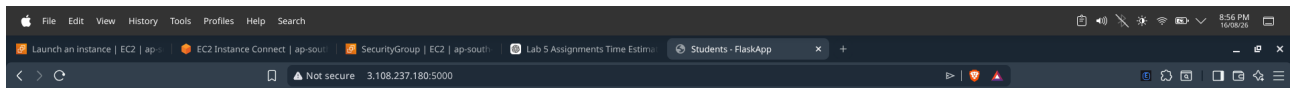
Step 14: Web Application Frontend - Delete

Initiated the deletion of the student record via the frontend confirmation prompt.



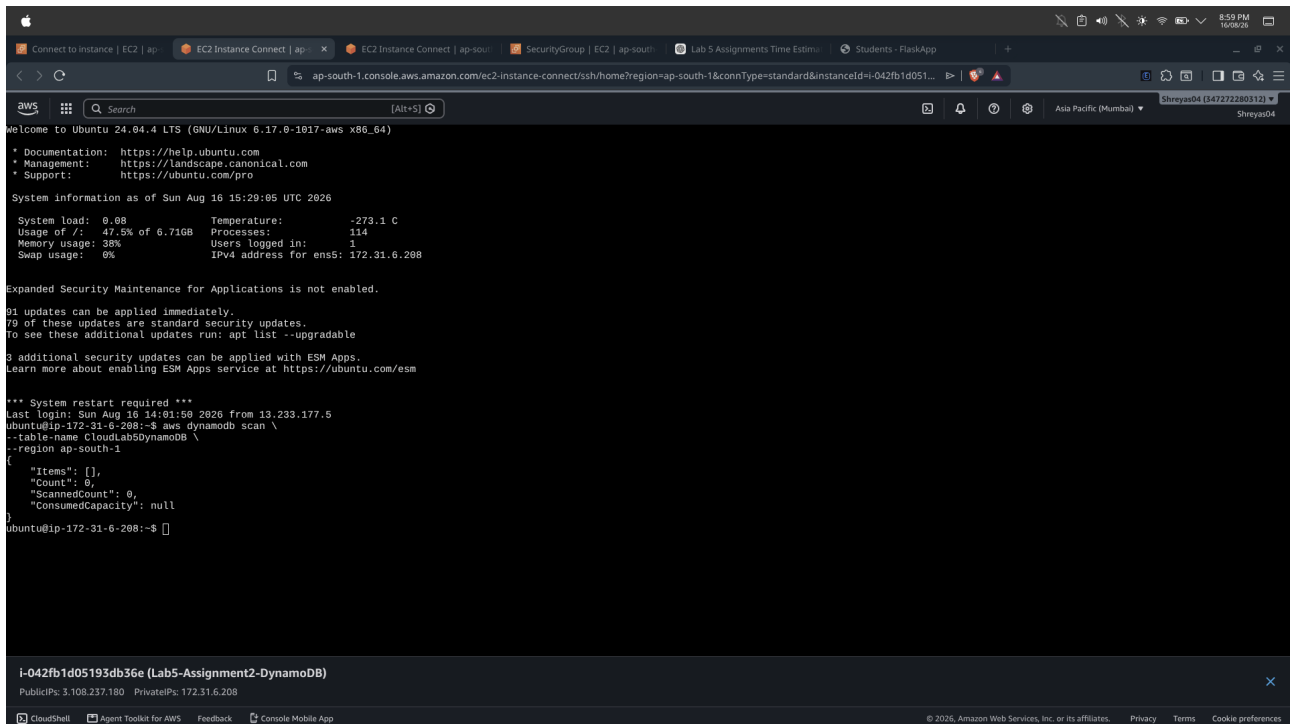
Step 15: Web Application Frontend - Deletion Verified

The student record was completely removed from the web interface.



Step 16: Backend Deletion Verification via CLI

A final 'aws dynamodb scan' verified that the table is fully empty, proving the frontend delete operation worked.



3. DATABASE DETAILS

Parameter	Value
Database Engine	AWS DynamoDB (NoSQL)
Table Name	CloudLab5DynamoDB
Partition Key	id (String)
Billing Mode	PAY_PER_REQUEST
Application Server	AWS EC2 (Ubuntu 24.04)
Web Framework	Python Flask
SDK	boto3
EC2 Public IP	3.108.237.180:5000

4. CRUD DEMONSTRATION

Operation	What was performed	Evidence
CREATE	Created student "Rohit Sharma", Age 22 via UI.	Refer to Steps 11 & 12
READ	Retrieved and displayed the student list in the UI.	Refer to Step 12
UPDATE	Updated Rohit Sharma's age to 23 years old via UI.	Refer to Step 13
DELETE	Deleted the student record via the frontend UI.	Refer to Steps 14 & 15

5. SECURITY CONFIGURATION

Security configuration was primarily managed using AWS IAM Roles. An IAM Role ('Lab5-Assignment2-DynamoDB-Role') was attached to the EC2 instance, allowing the Python application to authenticate and interact securely with the DynamoDB service without storing sensitive access keys in the code. Network access was demonstrated by successfully reaching the application on port 5000 over the internet.

6. RESULT

The Python Flask application running on AWS EC2 was successfully connected to an AWS DynamoDB NoSQL database. Full Create, Read, Update, and Delete operations were successfully demonstrated via the AWS CLI and the web interface.

7. CONCLUSION

The successful integration of the EC2-hosted Flask application with AWS DynamoDB proves the ability to utilize managed NoSQL cloud database services. The verified execution of CRUD operations confirms that the application can seamlessly utilize the AWS SDK (boto3) to securely read and write data to DynamoDB via IAM roles.

8. POST-LAB / ASSIGNMENT REQUIREMENTS

No additional Post-Lab task/question was specified for Assignment 2 in the provided lab document.

9. EVIDENCE SUMMARY

- ✓ EC2 application deployed and running
- ✓ AWS DynamoDB NoSQL table created
- ✓ IAM Role attached securely for Database connectivity
- ✓ CRUD – Create (CLI and UI)
- ✓ CRUD – Read (CLI and UI)
- ✓ CRUD – Update (CLI and UI)
- ✓ CRUD – Delete (CLI and UI)
- ✓ EC2 Application URL accessed (<http://3.108.237.180:5000>)

10. REFERENCES

1. Official Cloud Computing Lab 5 Assignment 2 Manual and Instruction Rubric.